Mini's n More — Technical Handover Pack

SSL, architecture, deployment, management, environment/runbook, integrations, logs and database

14 October 2026

- SSL certificate creation and usage
  - Scope
  - Prerequisites
  - First certificate creation
  - How the certificate is used
  - Verification after issue or Nginx change
  - Automatic renewal
  - Adding or removing a hostname
  - Troubleshooting
  - Renewal incident procedure
- Architecture diagram
  - Live process map
  - Ports on this host
  - Request examples
  - Data stores
  - Code layout
- Deployment guide
  - Layout on the VPS
  - What is running
  - Build artefacts
  - Restart one service (preferred)
  - Full isolated stack
  - Nginx
  - Laptop → server (optional)
  - After every deploy
- Management guide
  - Access boundary
  - Daily checks
  - PM2
  - Nginx / TLS
  - Admin operations
  - Payments and mail
  - What this VPS also hosts
  - Backup expectations
  - Change window
- Environment variables and runbook
  - Variable groups
    - Must be set or the stack will not boot
    - Public URLs
    - Mail
    - Social login
    - Object storage (Cloudflare R2)
    - Zoho
    - Other
    - XpressBees
  - Runbook — start of day / incident
  - Runbook — planned bounce (one service)
  - Runbook — env missing
- Zoho, XpressBees, and Mini’s n More admin integrations
  - Production callback URLs
  - Callback secrets and headers
  - What each Zoho webhook does
    - Shipment rules that matter operationally
  - Outbound Zoho (not webhooks)
  - XpressBees integration
    - Outbound calls made by Mini’s n More
    - Forward-shipment callback
    - Return/exchange callback
  - Mini’s n More admin integration operations
  - How to verify
  - Zoho console checklist
  - XpressBees console checklist
- Server logs and database — what to check
  - Need

    - Logs
        - What a healthy access line looks like
        - First places to grep
    - Postgres
        - Checks that matter
        - Backup (recommended)
    - Redis
    - Quick health matrix
- Database schema
    - Scope and source of truth
    - High-level ER diagram
    - Core commerce schema
        - `category`
        - `product`
        - `customer_order`
        - `order_item`
        - `coupon`
    - Returns, exchanges, and cancellation
    - Customer and notification tables
    - Authentication and roles
    - Content, operations, and configuration
    - Webhook idempotency and audit tables
    - Main enumerated values
    - Tables managed by infrastructure
    - Production verification queries

SSL certificate creation and usage

Scope

Production HTTPS terminates at Nginx for:

- `minisnmore.com`
- `www.minisnmore.com`

The certificate is issued by Let's Encrypt and managed by Certbot. Application services bind only to `127.0.0.1`; they do not load the public certificate themselves.

Expected certificate files:

```
/etc/letsencrypt/live/minisnmore.com/fullchain.pem
/etc/letsencrypt/live/minisnmore.com/privkey.pem
```

The `live` paths are Certbot-managed symlinks. Do not copy, rename, or manually replace files under `/etc/letsencrypt/live`, `archive`, or `renewal`.

Prerequisites

Before issuing a certificate:

1. DNS `A`/`AAAA` records for both names must resolve to this VPS. Remove a stale `AAAA` record if the VPS does not serve IPv6.
2. Inbound TCP ports 80 and 443 must be open.
3. Nginx must have a port-80 server block for both names.
4. No other process may occupy ports 80 or 443.
5. Run the commands as a sudo-capable server operator.

Read-only checks:

```
dig +short minisnmore.com A
dig +short www.minisnmore.com A
dig +short minisnmore.com AAAA
sudo ss -ltnp | grep -E ':(80|443)\b'
sudo nginx -t
```

First certificate creation

Install Certbot and its Nginx plugin if they are not already installed:

```
sudo apt update
sudo apt install -y certbot python3-certbot-nginx
```

Take a recoverable copy of the active Nginx site before allowing Certbot to edit it:

```
sudo cp -a /etc/nginx/sites-available/mini-prod.conf \
  /etc/nginx/sites-available/mini-prod.conf.pre-certbot
```

Issue one certificate containing both names:

```
sudo certbot --nginx \
  --cert-name minisnmore.com \
  -d minisnmore.com \
  -d www.minisnmore.com
```

Choose HTTP-to-HTTPS redirect when prompted. Certbot should add or maintain these directives in the HTTPS server block:

```
listen 443 ssl http2;
server_name minisnmore.com www.minisnmore.com;

ssl_certificate /etc/letsencrypt/live/minisnmore.com/fullchain.pem;
ssl_certificate_key /etc/letsencrypt/live/minisnmore.com/privkey.pem;
include /etc/letsencrypt/options-ssl-nginx.conf;
ssl_dhparam /etc/letsencrypt/ssl-dhparams.pem;
```

The port-80 block should redirect both names to HTTPS. Do not expose ports `3000` or `9180–9184` publicly as a workaround.

How the certificate is used

Nginx decrypts HTTPS and forwards HTTP to local services:

```
Browser/provider HTTPS :443
        ↓ TLS terminates here
Nginx
        ├── frontend 127.0.0.1:3000
        ├── gateway  127.0.0.1:9180
        └── selected callbacks 127.0.0.1:9184
```

Every proxy location must preserve the original scheme so OAuth redirects, secure cookies, and generated links remain HTTPS:

```
proxy_set_header Host $host;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto https;
proxy_set_header X-Forwarded-Port 443;
```

Certificate files contain private key material. Only Nginx/root should read the private key. Never commit or send `privkey.pem` through email or chat.

Verification after issue or Nginx change

```
sudo nginx -t
sudo systemctl reload nginx
sudo certbot certificates
curl -fsSI https://minisnmore.com/
curl -fsSI https://www.minisnmore.com/
openssl s_client -connect minisnmore.com:443 -servername minisnmore.com </dev/null 2>/dev/null \
  | openssl x509 -noout -subject -issuer -dates -ext subjectAltName
```

Acceptance criteria:

- Nginx configuration test passes.
- Both hostnames redirect or respond over HTTPS without a name warning.
- Subject Alternative Name contains both production hostnames.
- `notAfter` is in the future.

- `https://minisnmore.com/api/v1/categories` still returns JSON through the same TLS endpoint.

Automatic renewal

Ubuntu normally runs Certbot through a systemd timer. Confirm it is enabled and test renewal without changing the certificate:

```
systemctl list-timers --all | grep certbot
sudo systemctl status certbot.timer
sudo certbot renew --dry-run
```

Certbot renews certificates near expiry and reloads Nginx when the Nginx installer is recorded. Verify `/etc/letsencrypt/renewal/minisnmore.com.conf` references the expected certificate and authenticator, but do not place secret values in this handover pack.

Monthly operational check:

```
sudo certbot certificates
sudo journalctl -u certbot --since '35 days ago' --no-pager
```

Set an external expiry alert at least 21 days before expiry; automation should not be the only warning.

Adding or removing a hostname

Changing names reissues the certificate. Confirm DNS first, then specify the complete desired name set:

```
sudo certbot --nginx --cert-name minisnmore.com \
  -d minisnmore.com \
  -d www.minisnmore.com
```

Omitting an existing `-d` name can remove it from the renewed certificate. After a name change, also update Nginx `server_name`, DNS, CORS, OAuth redirect URLs, mail base URL, and any provider webhook URLs.

Troubleshooting

Symptom	Likely cause	Check / recovery
`NET::ERR_CERT_COMMON_NAME_INVALID`	Requested hostname is not in SAN	Inspect with `openssl`; reissue with both `-d` names
Certbot HTTP-01 challenge fails	DNS points elsewhere, port 80 blocked, or wrong Nginx site	Check `dig`, firewall/security group, `nginx -T`
Old certificate still served	Another HTTPS server block or CDN edge is answering	Compare origin IP, `nginx -T`, and `openssl -servername` output
Renewal succeeds but Nginx serves old dates	Nginx was not reloaded	`sudo nginx -t && sudo systemctl reload nginx`
Nginx fails after certificate edit	Bad path or unreadable key	Restore the saved site file, run `nginx -t`, then reload
Browser loops between HTTP and HTTPS	Conflicting proxy/CDN SSL mode or redirects	Check the CDN mode and Nginx port-80/443 blocks
Webhooks/OAuth fail after HTTPS change	Wrong callback URL or forwarded scheme	Confirm provider URL and `X-Forwarded-Proto https`

Useful logs:

```
sudo journalctl -u certbot -n 100 --no-pager
sudo journalctl -u nginx -n 100 --no-pager
sudo tail -n 100 /var/log/nginx/error.log
```

Do not use `--force-renewal` repeatedly: Let's Encrypt rate limits apply. Use `--dry-run` for routine testing.

Renewal incident procedure

If expiry is approaching and automatic renewal failed:

1. Confirm DNS and port 80 reach this server.
2. Run `sudo certbot renew --dry-run` and capture the exact failure.
3. Fix DNS, firewall, or Nginx validation errors.
4. Run `sudo certbot renew` once.
5. Run `sudo nginx -t && sudo systemctl reload nginx`.

6. Verify both names and the public API with the commands above.
7. Record the cause and restore the timer/alert so manual renewal is not required next time.

Revocation or private-key exposure is a security incident: issue a replacement key/certificate, revoke the affected certificate with Certbot, reload Nginx, and rotate any copied key material.

Architecture diagram

Live production (this VPS) does **not** use Consul. Nginx talks to a local isolated stack on **9180–9184**. Catalog core is not public.

Public site: `https://minisnmore.com` and `https://www.minisnmore.com`
TLS: Let’s Encrypt (`/etc/letsencrypt/live/minisnmore.com/`)
Nginx site: `/etc/nginx/sites-available/mini-prod.conf`
Locations: `deploy/nginx/prod.minisnmore.com.isolated-9180.locations.conf`

Live process map

```
flowchart LR
  Browser["Browser"]
  Nginx["Nginx :443"]
  FE["Frontend PM2<br/>mini-prod-frontend :3000"]
  GW["Gateway<br/>mini-prod-ms-gateway :9180"]
  ST["Store<br/>mini-prod-ms-store :9181"]
  CM["Commerce<br/>mini-prod-ms-commerce :9182"]
  CC["Catalog core<br/>mini-prod-ms-catalog-core :9184"]
  PG[("Postgres miniandmore :5432")]
  R2["Cloudflare R2"]
  Zoho["Zoho Books"]
  XB["XpressBees"]
  Rzp["Razorpay"]
  Mail["SMTP Gmail or Zoho"]

  Browser --> Nginx
  Nginx -->|HTML /assets| FE
  Nginx -->|"/api /oauth2 /login/oauth2"| GW
  Nginx -->|"invoice webhooks"| CC
  GW -->|shop CMS admin catalog| ST
  GW -->|orders coupons returns payments| CM
  ST -->|/internal/v1| CC
  CM -->|/internal/v1| CC
  CC --> PG
  GW --> PG
  CC --> R2
  CC --> Zoho
  CC --> XB
  CC --> Rzp
  GW --> Mail
```

Ports on this host

Process	PM2 name	Bind	Role
Storefront + admin UI	`mini-prod-frontend`	`127.0.0.1:3000`	Vite preview of `dist/`
Gateway	`mini-prod-ms-gateway`	`127.0.0.1:9180`	JWT, OAuth, route `/api`
Store	`mini-prod-ms-store`	`127.0.0.1:9181`	Catalog/CMS/admin proxy
Commerce	`mini-prod-ms-commerce`	`127.0.0.1:9182`	Orders, coupons, returns, payments
Catalog core	`mini-prod-ms-catalog-core`	`127.0.0.1:9184`	Postgres, Zoho, Razorpay, R2
Integration	*(not running)*	`:9183` if started	Optional Zoho helper; catalog can call it

Gateway Spring profile on the live JAR: `prod,uat-isolated` (profile **name** is historical; routes point at 9181/9182/9184).

Request examples

Browser path	Nginx	Then
`/`, `/shop`, `/admin`, `/product/...`	`:3000`	SPA
`/api/authenticate`, `/api/register`	`:9180`	Gateway itself
`/api/v1/categories`, `/api/v1/products`, `/api/v1/admin/products`	`:9180` → store `:9181` → catalog `:9184`	
`/api/v1/orders`, `/api/v1/payments`, `/api/v1/coupons`, `/api/v1/order-returns`	`:9180` → commerce `:9182` → catalog `:9184`	
`/oauth2/`, `/login/oauth2/`	`:9180`	Google/Apple/Facebook
`/api/v1/zoho/invoice-webhook`	**direct `:9184`**	Catalog (not via gateway)
`/api/v1/zoho/shipment-webhook` (and stock, item, order-status)	`:9180`	Gateway route → catalog `:9184`
`/api/v1/webhooks/xpressbees/shipments` and `/returns`	`:9180`	Gateway route → catalog `:9184`

Shared JWT secret: `JHIPSTER_SECURITY_AUTHENTICATION_JWT_BASE64_SECRET` in `/etc/mini-and-more/prod.env`. Every Java service must use the same value.

Data stores

Store	Use
Postgres `miniandmore` on `127.0.0.1:5432`	Catalog **and** gateway users on this host (both JDBC and R2DBC)
Redis `127.0.0.1:6379`	Catalog cache (`JHIPSTER_CACHE_REDIS_SERVER`)
Cloudflare R2 bucket `miniandmore`, prefix `prod`	Product images
Other `ledgerly_*` Postgres DBs	Separate Ledgerly product on this VPS — **not** Mini’s n More checkout

Consul is **disabled** on the isolated stack (`--spring.cloud.consul.enabled=false`). Runtime Liquibase is **off**; schema is applied with `deploy/scripts/run-catalog-liquibase.sh` before a catalog JAR bounce.

Code layout

Folder	Service
`src/`	Storefront + admin (TanStack Start / React)
`gateway/`	Auth + routing
`store/`	Store edge
`commerce/`	Commerce edge
`backend/`	Catalog core (`miniandmore`)
`integration/`	Optional; not in the current PM2 list
`deploy/`	Scripts, Nginx snippets, env example

Companion architecture write-ups: `docs/system-architecture.md`, `docs/backend-architecture.md`, `docs/frontend-architecture.md`, `docs/database-er-diagram.md`. Those still mention 8080/Consul in places; **this handover diagram is what is actually live**.

Deployment guide

This host is **production**. Public traffic is Nginx → frontend `:3000` and API gateway `:9180`.

Do not point scripts at `uat.minisnmore.com` or `/etc/mini-and-more/uat.env`. Use `/etc/mini-and-more/prod.env`.

Layout on the VPS

Item	Value
App root	`/var/www/uat1/mini_ms_test` (path name is historical; this **is** prod)
Env file	`/etc/mini-and-more/prod.env` (`chmod 600`)
Nginx	`/etc/nginx/sites-available/mini-prod.conf` → `sites-enabled/mini-prod.conf`
API snippet	`deploy/nginx/prod.minisnmore.com.isolated-9180.locations.conf`
Gateway JAR used by PM2	`deploy/gateway-prod.jar`
Git	`origin` = `https://github.com/developersqobox-org/mini_and_more.git`, branch `prod`

What is running

```
mini-prod-frontend          :3000
mini-prod-ms-gateway        :9180
mini-prod-ms-store          :9181
mini-prod-ms-commerce       :9182
mini-prod-ms-catalog-core   :9184
```

`pm2 save` writes `/root/.pm2/dump.pm2` so processes come back after reboot (`pm2 startup` should already be enabled).

Build artefacts

Service	Typical JAR / output
Catalog	`backend/target/miniandmore-*.jar`
Store	`store/target/miniandmore-store-*.jar`
Commerce	`commerce/target/miniandmore-commerce-*.jar`
Gateway	`gateway/target/gateway-*.jar` copied to `deploy/gateway-prod.jar`
Frontend	`npm ci && npm run build` → `dist/`

Java: `./mvnw -Pprod -DskipTests package` in each module. Runtime Liquibase is disabled; apply schema **before** swapping the catalog process:

```
# uses SPRING_DATASOURCE_* from prod.env
set -a && . /etc/mini-and-more/prod.env && set +a
bash /var/www/uat1/mini_ms_test/deploy/scripts/run-catalog-liquibase.sh \
  /var/www/uat1/mini_ms_test/backend/target/miniandmore-0.0.1-SNAPSHOT.jar
```

Restart one service (preferred)

Always wait for `/management/health/liveness` before moving on.

Service	Script
Catalog	`bash deploy/scripts/prod-restart-isolated-catalog.sh`
Commerce	`bash deploy/scripts/prod-restart-isolated-commerce.sh`
Gateway	`bash deploy/scripts/prod-restart-isolated-gateway.sh`
Frontend (rebuild)	`bash deploy/scripts/prod-frontend-pm2.sh`
Frontend (`dist/` already built)	`bash deploy/scripts/prod-frontend-restart.sh`
Reload env into running JVMs	`bash deploy/scripts/prod-reload-env-and-restart.sh`

Store has no dedicated helper in all cases; start it the same way as commerce (prod profile, `APPLICATION_CATALOG_BASE_URL=http://127.0.0.1:9184`, port `9181`) or use the isolated stack script.

Scripts delete **both** `mini-prod-ms-*` and legacy `mini-uat-ms-*` names so a bounce cannot leave two processes on the same port.

Full isolated stack

```
REPO_ROOT=/var/www/uat1/mini_ms_test bash deploy/scripts/prod-microservices-isolated-pm2.sh
```

This recycles **9180–9184 only**. It does not touch `:3000`. Boot can take several minutes per JVM. Health check:

```
GATEWAY_URL=http://127.0.0.1:9180 \
STORE_URL=http://127.0.0.1:9181 \
COMMERCE_URL=http://127.0.0.1:9182 \
CATALOG_CORE_URL=http://127.0.0.1:9184 \
  bash deploy/scripts/health-check-microservices.sh
```

(The health script also probes integration `:9183`. If integration is not deployed, that check fails even when the shop is fine.)

Nginx

```
nginx -t && systemctl reload nginx
```

Public API must keep proxying `/api/` to `127.0.0.1:9180` except invoice webhooks, which go to `:9184` (see zoho-webhooks-and-details.md).

Laptop → server (optional)

`deploy/scripts/prod-deploy-frontend-from-laptop.sh` and `prod-deploy-gateway-from-laptop.sh` rsync build output then run the restart scripts on the VPS. Confirm `REMOTE_DIR=/var/www/uat1/mini_ms_test`.

After every deploy

1. `curl -fsS https://minisnmore.com/api/v1/categories` → JSON `200`
2. Homepage and `/shop` load products
3. `pm2 list` — all `mini-prod-*` **online**, no `errored`
4. `pm2 save`

If a health check fails, keep the prior JAR or `dist/` available and restore only the affected service. Re-run the same health checks before reopening the change window.

Management guide

How to operate Mini's n More production without changing code.

Access boundary

Routine operation may require existing authorised access to the VPS, Nginx/PM2/Postgres, Zoho, Razorpay, OAuth provider consoles, DNS, and object storage. Account ownership transfer is outside this document's scope.

Do not share the contents of `/etc/mini-and-more/prod.env` over chat. Operators should retrieve secrets from the approved vault and keep access least-privileged.

Daily checks

```
pm2 list
curl -fsS -o /dev/null -w '%{http_code}\n' https://minisnmore.com/
curl -fsS -o /dev/null -w '%{http_code}\n' https://minisnmore.com/api/v1/categories
df -h /
free -h
```

Expect `200` on home and categories. Shop may `307` then `200` on `/shop?sort=new`. Unauthenticated `/api/v1/order-returns` should be `401`, not `502`/`503`.

PM2

```
pm2 list
pm2 logs mini-prod-ms-gateway --lines 80
pm2 logs mini-prod-ms-catalog-core --lines 80
pm2 restart mini-prod-ms-store --update-env   # only after sourcing prod.env
```

A plain `pm2 restart` **does not** pick up new env file values. Use `deploy/scripts/prod-reload-env-and-restart.sh`.

Do not `pm2 delete` the frontend during business hours unless you immediately start `mini-prod-frontend` again on `:3000`.

Nginx / TLS

```
nginx -t
systemctl status nginx
certbot certificates
```

Site file: `/etc/nginx/sites-available/mini-prod.conf`. It includes the isolated locations conf from the repo. After editing the snippet:

```
nginx -t && systemctl reload nginx
```

Admin operations

- Catalog edits: https://minisnmore.com/admin/products
- Orders / returns: `/admin/orders`, `/admin/returns`
- Zoho: `/admin/zoho` — long timeouts; wait for the UI result

Storewide discount lives on **Discount**. Changing it recalculates selling price from MRP on simple products.

Payments and mail

- Razorpay: live key id is in `prod.env` (`RAZORPAY_KEY_ID`). Demo flags `RAZORPAY_PAYMENT_DEMO` / `RAZORPAY_REFUND_DEMO` must stay **false** in production.
- Mail: `SPRING_MAIL_PROVIDER` is `gmail` or `zoho`. From address must match the authenticated mailbox.

What this VPS also hosts

`ledgerly-*` Postgres databases and a stopped `ledgerly-frontend` PM2 app belong to a **different** product. Do not drop those databases or reuse their users for Mini's n More.

Backup expectations

What	Where	Operator action
App code	GitHub `prod` branch	Push before every server-only hotfix
Env / secrets	`/etc/mini-and-more/prod.env`	Off-server copy in a vault
Postgres `miniandmore`	local 5432	Schedule `pg_dump` (see server-logs-and-database.md)
Images	R2 bucket `miniandmore` prefix `prod`	Cloudflare versioning / second bucket
Nginx	`/etc/nginx` + repo `deploy/nginx`	Keep repo copy in sync

Change window

Prefer catalog/gateway bounces off peak. Each JVM can take 30s–several minutes. Nginx keeps serving the old frontend until `mini-prod-frontend` is restarted.

Environment variables and runbook

File on the server: `/etc/mini-and-more/prod.env` (`chmod 600`, root only)
Template in git: `deploy/env/prod.env.example`
Do not commit real values.

Java processes do not re-read this file until they are started with `--update-env` (the prod restart scripts do that).

Live isolated stack **overrides** some example defaults:

Variable	Live isolated value
APPLICATION_CATALOG_BASE_URL	http://127.0.0.1:9184
MINIANDMORE_STORE_URI	http://127.0.0.1:9181
MINIANDMORE_COMMERCE_URI	http://127.0.0.1:9182
MINIANDMORE_INTEGRATION_URI	http://127.0.0.1:9183
SPRING_DATASOURCE_URL	jdbc:postgresql://127.0.0.1:5432/miniandmore
SPRING_R2DBC_URL	r2dbc:postgresql://127.0.0.1:5432/miniandmore
S3_KEY_PREFIX	prod
VITE_API_BASE_URL / mail / CORS / Zoho OAuth	https://minisnmore.com
LOG_JSON	false on this host (JSON encoder not on all classpaths)
GATEWAY_RESPONSE_TIMEOUT	600s (Zoho admin)

`prod.env.example` still shows `:8084` and a separate `gateway` database. That is the **8080 layout**. This VPS uses the **918x isolated** layout above.

Variable groups

Must be set or the stack will not boot

Name	Used by
JHIPSTER_SECURITY_AUTHENTICATION_JWT_BASE64_SECRET	All Java services (same value)
INTERNAL_API_KEY	Catalog ↔ integration / internal admin
SPRING_DATASOURCE_*	Catalog JDBC
SPRING_R2DBC_* / SPRING_LIQUIBASE_*	Gateway users (same DB on this host)
RAZORPAY_KEY_ID / RAZORPAY_KEY_SECRET	Catalog prod validator

Public URLs

`JHIPSTER_MAIL_BASE_URL`, `FRONTEND_AUTH_CALLBACK_URL`, `CORS_ALLOWED_ORIGIN_PATTERNS`, `ZOHO_OAUTH_*`, `VITE_API_BASE_URL`.

Mail

`SPRING_MAIL_PROVIDER` = `gmail` or `zoho`. Provider-specific `GMAIL_MAIL_*` / `ZOHO_MAIL_*` are copied onto `SPRING_MAIL_*` by `deploy/scripts/lib/gateway-env.sh`.

Social login

`GOOGLE_*`, `APPLE_*`, `FACEBOOK_*`. Empty values are replaced with a placeholder at gateway start so the JVM boots; that provider's login will fail until real ids are set.

Object storage (Cloudflare R2)

`S3_ENDPOINT`, `S3_BUCKET`, `S3_ACCESS_KEY`, `S3_SECRET_KEY`, `S3_PUBLIC_BASE_URL`, `S3_KEY_PREFIX=prod`, `S3_MAX_UPLOAD_BYTES`.

Zoho

`ZOHO_CLIENT_ID` / `SECRET` / `ORG_ID` / `DATA_CENTER=in`, sync flags, `ZOHO_*_WEBHOOK_SECRET` (see zoho-webhooks-and-details.md).

Other

`LEDGERLY_*` — present in env; catalog can run Ledgerly beside Zoho.
`FASTSMS_*` — OTP SMS.

XpressBees

Name	Purpose
XPRESSBEES_BASE_URL	Carrier API base URL
XPRESSBEES_EMAIL, XPRESSBEES_PASSWORD	Token-login credentials
XPRESSBEES_XB_KEY	Legacy/key-based API credential when the configured auth mode requires it
XPRESSBEES_WEBHOOK_SECRET	HMAC/shared secret for shipment and return callbacks
XPRESSBEES_WAREHOUSE_*	Pickup/contact address sent on shipment creation
XPRESSBEES_ORIGIN_PINCODE	Origin used for serviceability and shipment requests
XPRESSBEES_PICKUP_VENDOR_CODE	Provider pickup location identifier, when assigned

Changing courier credentials or webhook secret requires a catalog restart. Never put the real values in `deploy/env/prod.env.example`.

Runbook — start of day / incident

1. `pm2 list` — all `mini-prod-ms-*` and `mini-prod-frontend` **online**.
2. Hit `http://127.0.0.1:9180/management/health/liveness` and `:9181`, `:9182`, `:9184`.
3. Hit `https://minisnmore.com/api/v1/categories`.
4. If API is 502/504: store or catalog is down — restart catalog first, then store, then gateway.
5. If HTML is 502 but API works: restart `mini-prod-frontend` only.
6. If OAuth fails: check `GOOGLE_CLIENT_*` and callback `https://minisnmore.com/auth/callback`.
7. After editing `prod.env`:
   `bash /var/www/uat1/mini_ms_test/deploy/scripts/prod-reload-env-and-restart.sh`

Runbook — planned bounce (one service)

Catalog first if the JAR or schema changed, then store, commerce, gateway. Frontend last if `dist/` changed.

```
cd /var/www/uat1/mini_ms_test
bash deploy/scripts/prod-restart-isolated-catalog.sh
bash deploy/scripts/prod-restart-isolated-commerce.sh
bash deploy/scripts/prod-restart-isolated-gateway.sh
```

Wait for `OK` in each script. Gateway often needs ~30s–3 min.

Runbook — env missing

If `/etc/mini-and-more/prod.env` is gone, restore from vault. A leftover `/etc/mini-and-more/uat.env` may still exist as a **backup copy** from the rename; new scripts do not source it. Prefer `prod.env` only.

Zoho, XpressBees, and Mini’s n More admin integrations

This document separates three related integration surfaces:

1. Zoho Books / Inventory callbacks into Mini’s n More.
2. XpressBees forward-shipment and reverse-pickup callbacks into Mini’s n More.
3. Authenticated Mini’s n More admin actions that call those providers.

Inbound callbacks are not admin APIs. Callback URLs are unauthenticated at the JWT layer so providers can reach them, but every callback is authenticated with a provider-specific HMAC or shared secret. Admin operations require an authenticated `ROLE_ADMIN` or `ROLE_SUPER_ADMIN` session.

Secrets live only in `/etc/mini-and-more/prod.env`. Do not paste them here.

Full payload examples and workflow criteria: `docs/zoho-webhook-setup-and-purpose.md`.

Production callback URLs

All HTTPS on https://minisnmore.com.

Use the `/api/...` URL as the canonical provider-facing address. `/internal/...` aliases exist for compatibility and internal routing; do not configure a new provider with them.

Provider / event	Method	Canonical URL	Route after Nginx
Zoho invoice status	POST	`/api/v1/zoho/invoice-webhook`	Direct to catalog `:9184`
Zoho shipment	POST	`/api/v1/zoho/shipment-webhook`	Gateway `:9180` → catalog `:9184`
Zoho sales-order status	POST	`/api/v1/zoho/order-status-webhook`	Gateway `:9180` → catalog `:9184`
Zoho item / catalog	POST	`/api/v1/zoho/item-webhook`	Gateway `:9180` → catalog `:9184`
Zoho stock	POST	`/api/v1/zoho/stock-webhook`	Gateway `:9180` → catalog `:9184`
XpressBees forward shipment	POST	`/api/v1/webhooks/xpressbees/shipments`	Gateway `:9180` → catalog `:9184`
XpressBees return / exchange	POST	`/api/v1/webhooks/xpressbees/returns`	Gateway `:9180` → catalog `:9184`

Invoice is special: Nginx proxies it **straight to catalog core** so a gateway restart cannot drop paid/void events.

Admin Zoho UI (OAuth, bulk sync) uses `/api/v1/admin/zoho/` with a **600s** proxy timeout.

Callback secrets and headers

Callback	Environment variable	Accepted shared-secret header
Invoice	`ZOHO_INVOICE_WEBHOOK_SECRET`	`X-Zoho-Invoice-Secret`
Shipment	`ZOHO_SHIPMENT_WEBHOOK_SECRET`	`X-Zoho-Shipment-Secret`
Order status	`ZOHO_ORDER_STATUS_WEBHOOK_SECRET`	`X-Zoho-Order-Status-Secret`
Item	`ZOHO_ITEM_WEBHOOK_SECRET`	`X-Zoho-Item-Secret`
Stock	`ZOHO_STOCK_WEBHOOK_SECRET`	`X-Zoho-Stock-Secret`
XpressBees shipment and return	`XPRESSBEES_WEBHOOK_SECRET`	`X-XpressBees-Secret`

Zoho may instead send its event-specific signature header or `X-Zoho-Webhook-Signature`. XpressBees may instead send `X-Hmac-SHA256`, `X-XpressBees-Signature`, or `X-XpressBees-Webhook-Signature`. MiniMS verifies the HMAC against the exact raw request body; middleware must not reformat the JSON before verification.

Secret rules (Zoho): 12–50 characters, alphanumeric only, no punctuation.

A `401` from MiniMS means the header does not match `prod.env`. Restart catalog (and gateway for `/internal` routes) after changing secrets.

What each Zoho webhook does

Event	MiniMS effect
Invoice created/edited (Paid / Sent / Void)	Local invoice id, payment/void state, PDF cache opportunity
Shipment created/edited	Tracking, “on the way”, delivered — **this is the delivery source of truth**
Sales order status (Invoiced / Void / Cancelled / Fulfilled)	Accounting state. **`fulfilled` does not mark the store order delivered**
Item changed	One product/SKU refresh instead of a full catalog pull
Stock	Absolute `stock_on_hand` for that SKU (variants need `sku`)

Shipment rules that matter operationally

Three Zoho Inventory workflow rules should all call the **same** shipment webhook:

1. Package created (package slip not empty) → MiniMS can show packed / hand-wrapped.
2. Status **Shipped** → on the way (not delivered).
3. Status **Fulfilled** (or Delivered) → MiniMS delivered.

Do not invent a MiniMS custom field to fake delivery. A later sales-order “fulfilled” from invoicing must not undo a shipment already marked delivered.

Payload **must** include `salesorder_id` or `salesorder_number` so MiniMS can find the local order. Default Zoho preview JSON is not enough.

`eventId` is used for idempotency. Duplicate posts return `200` and are ignored.

Outbound Zoho (not webhooks)

Configured in `/admin/zoho` and env:

- OAuth: `ZOHO_CLIENT_ID`, `ZOHO_CLIENT_SECRET`, `ZOHO_ORG_ID`, `ZOHO_DATA_CENTER=in`
- Redirect: `https://minisnmore.com/admin/zoho` (must match the Zoho API console)
- Flags such as `ZOHO_AUTO_SYNC_ENABLED`, `ZOHO_ORDER_TRACKING_ENABLED`

If those flags are `false`, schedulers and order-page tracking pulls stop. **Webhooks still work.** Use that only when testing whether Zoho actually POSTs.

XpressBees integration

Outbound calls made by Mini’s n More

The catalog service logs in to `XPRESSBEES_BASE_URL` with the configured credentials. Admin order and return actions can:

- check courier serviceability/rates;
- create a forward shipment and store its AWB/tracking URL;
- cancel an active forward shipment;
- create a reverse pickup for an approved return or exchange; and
- record a manual carrier instead when XpressBees is not used.

The core settings are `XPRESSBEES_EMAIL`, `XPRESSBEES_PASSWORD`, `XPRESSBEES_XB_KEY`, `XPRESSBEES_WEBHOOK_SECRET`, origin/warehouse values, and pickup vendor code. Defaults and the complete non-secret key list live in `deploy/env/prod.env.example` and `backend/src/main/resources/config/application.yml`.

Forward-shipment callback

`POST /api/v1/webhooks/xpressbees/shipments` accepts either:

- an order reference: `order_number`, `order_no`, `OrderNo`, or `reference`; or
- an AWB: `awb`, `awb_number`, `AWBNumber`, or `AirWayBillNO`.

It also requires a carrier status in `status`, `shipment_status`, `event`, or `milestone`. The handler updates the current order’s shipment status. A delayed event for an old AWB after rebooking is acknowledged as `skipped: true` and cannot overwrite the new shipment.

Return/exchange callback

`POST /api/v1/webhooks/xpressbees/returns` accepts a return reference such as `return_number`, `order_no`, or `reference`, or finds the return by its reverse/outbound AWB. It updates reverse pickup progress and, for exchanges, outbound replacement progress. Delivered and out-for-delivery events can advance the local workflow and trigger the corresponding once-only notification.

Both handlers are idempotent. Providers should send a stable event id. When none is supplied, MiniMS derives

one from the local reference, normalized status, and AWB. Replayed completed events return `200` without applying the transition twice.

Mini’s n More admin integration operations

The admin UI is https://minisnmore.com/admin. These are browser/API operations initiated by an authorised administrator, not webhooks.

Admin area	Main operations	Public API family
Zoho	Connect OAuth, test connection, sync/pull products, pull stock, retry/sync orders, pull tracking	`/api/v1/admin/zoho/**`
Zoho settings	Read/update the stored integration settings	`/api/v1/admin/zoho-settings`
Orders	Create package, book/cancel XpressBees or manual shipment	`/api/v1/admin/orders/**`
Returns/exchanges	Approve/reject, book reverse pickup, receive/QC/refund, create replacement shipment	`/api/v1/admin/order-returns/**`

Operational rules:

1. Use the UI for normal work; do not invoke admin endpoints from a provider dashboard.
2. A Zoho bulk operation can take several minutes. Nginx and the gateway allow 600 seconds for `/api/v1/admin/zoho/**`.
3. Do not repeat a button click only because the browser appears slow. Check logs and the provider/local record first.
4. Confirm package weight, dimensions, items, address, payment type, and serviceability before booking XpressBees.
5. Do not manually move an order/return past a carrier-confirmed state unless the provider dashboard and local audit trail have been reconciled.
6. Admin requests use the logged-in JWT/cookie and role checks. Callback secrets do not grant admin access.

How to verify

```
# Nginx saw provider callbacks
grep -Ei 'zoho|xpressbees' /var/log/nginx/access.log | tail -50

# Catalog / gateway logs
pm2 logs mini-prod-ms-catalog-core --lines 100
pm2 logs mini-prod-ms-gateway --lines 100
```

HTTP	Meaning
200	Accepted or duplicate `eventId`
400	Bad JSON or missing required fields
401	Secret mismatch
404	No matching MiniMS order, return, AWB, or product
503	Secret missing in env, or catalog down

Postgres tables (in `miniandmore`):

```
zoho_invoice_webhook_event
zoho_shipment_webhook_event
zoho_order_status_webhook_event
zoho_item_webhook_event
zoho_stock_webhook_event
xpressbees_shipment_webhook_event
xpressbees_return_webhook_event
```

For a safe authentication test, send valid JSON with an intentionally invalid secret. The expected result is `401`; this proves DNS, TLS, Nginx, gateway/catalog routing, and endpoint registration without changing an order. Do not paste the real secret into shell history. Run a successful end-to-end test only with a dedicated test order/return and a provider-generated event.

Zoho console checklist

1. API console app: production redirect `https://minisnmore.com/admin/zoho`.
2. Workflow **Actions** (webhooks) created with compact JSON payloads (placeholders must not wrap across lines).

3. Workflow **Rules** on Invoices, Shipment Orders, Sales Orders, Items.
4. Org id matches `ZOHO_ORG_ID`.
5. After OAuth, `/admin/zoho` shows connected; do not full-sync catalog during peak.

XpressBees console checklist

1. Production account/API URL matches `XPRESSBEES_BASE_URL` and the configured authentication mode.
2. Warehouse address, origin pincode, phone, and pickup vendor code are complete.
3. Forward callback is `https://minisnmore.com/api/v1/webhooks/xpressbees/shipments`.
4. Reverse/return callback is `https://minisnmore.com/api/v1/webhooks/xpressbees/returns`.
5. The provider and `XPRESSBEES_WEBHOOK_SECRET` use the same HMAC/shared secret.
6. A test forward shipment and reverse pickup each produce an AWB, a `200` callback, and a row in the matching audit table.

Server logs and database — what to check

Need

You **do** need log + database access for production support:

- PM2 / Nginx logs: why a request 502'd, why Zoho returned 401, why JWT failed.
- Postgres `miniandmore`: catalog, orders, coupons, webhook event tables, **and** gateway users on this host.

You do **not** need the extra `gateway` database that still exists on this VPS — live R2DBC points at `miniandmore`. You do **not** need `ledgerly_*` databases for Mini's n More.

Logs

Source	Path / command	When
PM2 stdout/err	`/root/.pm2/logs/mini-prod-*.log`	JVM crash, Spring exceptions
Follow live	`pm2 logs mini-prod-ms-catalog-core --lines 200`	Deploy / webhook test
Nginx access	`/var/log/nginx/access.log`	Did Zoho / Razorpay / the browser hit us?
Nginx error	`/var/log/nginx/error.log`	Upstream timeout, SSL, 502
systemd nginx	`journalctl -u nginx -n 50`	Reload failures
Certbot	`journalctl -u certbot` / `certbot certificates`	TLS expiry

Frontend logs are mostly the Vite preview process (`mini-prod-frontend`). SPA errors also appear in the **browser console**.

`LOG_JSON=false` on this host so PM2 files stay line-oriented.

What a healthy access line looks like

```
GET /api/v1/categories → 200
GET /shop → 307 or 200
POST /api/v1/zoho/invoice-webhook → 200
POST /internal/v1/zoho/shipment-webhook → 200
POST /api/v1/webhooks/xpressbees/shipments → 200
POST /api/v1/webhooks/xpressbees/returns → 200
```

`401` on `/api/v1/order-returns` without a cookie is expected.

First places to grep

```
# Catalog / Zoho
grep -iE 'zoho|webhook|ERROR' /root/.pm2/logs/mini-prod-ms-catalog-core-out.log | tail -50

# Auth
grep -iE 'jwt|oauth|ERROR' /root/.pm2/logs/mini-prod-ms-gateway-out.log | tail -50

# Payments
grep -iE 'razorpay|ERROR' /root/.pm2/logs/mini-prod-ms-catalog-core-out.log | tail -50

# XpressBees forward/reverse callbacks
grep -iE 'xpressbees|expressbees|webhook|ERROR' /root/.pm2/logs/mini-prod-ms-catalog-core-out.log | tail
```

Disk: if `/` fills up, PM2 logs are the usual cause. Rotate or truncate old `~/.pm2/logs/*` after copying an incident snippet. Do not delete `/var/log/nginx` without logrotate.

Postgres

Database	Port	Used by live MiniMS?
`miniandmore`	5432	**Yes** — catalog JDBC + gateway R2DBC
`gateway`	5432	**No** (leftover; do not drop until confirmed unused)
`ledgerly_*`	5432	**No** — other product on this VPS

Connect using `SPRING_DATASOURCE_USERNAME` / `PASSWORD` from `prod.env` (catalog) or `SPRING_R2DBC_*` (same DB here).

Checks that matter

```
-- catalog up
SELECT count(*) FROM product;
SELECT count(*) FROM customer_order;

-- recent webhook processing
SELECT * FROM zoho_shipment_webhook_event ORDER BY id DESC LIMIT 20;
SELECT * FROM zoho_invoice_webhook_event ORDER BY id DESC LIMIT 20;
SELECT * FROM xpressbees_shipment_webhook_event ORDER BY id DESC LIMIT 20;
SELECT * FROM xpressbees_return_webhook_event ORDER BY id DESC LIMIT 20;

-- stuck orders (adjust names if schema differs)
SELECT id, order_number, status, created_date
FROM customer_order
ORDER BY created_date DESC
LIMIT 20;
```

Liquibase: runtime auto-migrate is **off**. Pending changesets are applied with `deploy/scripts/run-catalog-liquibase.sh` **before** swapping the catalog JAR. After a successful run you should see **0 pending**.

Backup (recommended)

```
# run as postgres or with prod.env credentials
pg_dump -Fc -h 127.0.0.1 -U "$SPRING_DATASOURCE_USERNAME" -d miniandmore \
  -f "/var/backups/miniandmore-$(date +%F).dump"
```

Store dumps **off this VPS**. Restore only onto a test copy of `miniandmore`, never onto `ledgerly_*`.

Images are **not** in Postgres; they are in Cloudflare R2 (`S3_BUCKET` + prefix `prod`). A DB restore without R2 still has broken product photos.

Redis

`127.0.0.1:6379` — JHipster cache for catalog. If Redis is down, catalog may still boot depending on config but will be slower / error on cache. Check: `redis-cli ping` → `PONG`.

Quick health matrix

Symptom	Check
Blank shop / 502 HTML	`pm2 list` frontend; nginx error log
502 on `/api`	gateway + store + catalog liveness; catalog logs
Login works, shop empty	catalog DB `product` / `active`; R2 URLs
Checkout pays but order missing	catalog `customer_order`; Razorpay dashboard; commerce logs
Zoho tracking never updates	Nginx POST to shipment webhook; `zoho_shipment_webhook_event`; Zoho workflow history
XpressBees tracking never updates	Nginx POST to the shipment/return callback; matching `xpressbees_*_webhook_event`; active AWB
Invoice PDF missing	invoice webhook 200; catalog `:9184` was up
Admin Zoho timeout	nginx 600s location; catalog still running the sync

Database schema

Scope and source of truth

This document describes the PostgreSQL schema represented by:

- `backend/src/main/resources/config/liquibase/master.xml` and its included changelogs;
- the current JPA entity mappings under `backend/src/main/java/com/miniandmore/app/domain`; and
- the production topology documented for the isolated `9180–9184` stack.

Snapshot date: **14 October 2026**. The newest included migration is `20261014120000_add_package_short_ship_qty.xml`.

The live isolated deployment points catalog JDBC and gateway R2DBC at the same `miniandmore` database on PostgreSQL `127.0.0.1:5432`. The repository template still contains older examples with a separate `gateway` database; those are not the documented live layout.

This is a structural document, not a data export. It contains no credentials or customer records. Because the production database was not reachable while this document was generated, operators must use the verification queries at the end to confirm that every Liquibase change has been applied.

High-level ER diagram

Solid relationships below are enforced by foreign keys. Dotted relationships are logical references stored as ids, codes, or login strings without a database foreign key.

```
erDiagram
    JHI_USER ||--o{ JHI_USER_AUTHORITY : user_id
    JHI_AUTHORITY ||--o{ JHI_USER_AUTHORITY : authority_name

    CATEGORY ||--o{ PRODUCT : category_id
    CUSTOMER_ORDER ||--o{ ORDER_ITEM : order_id
    CUSTOMER_ORDER ||--o{ ORDER_RETURN : customer_order_id
    CUSTOMER_ORDER ||--o{ ORDER_CANCELLATION_REQUEST : customer_order_id
    ORDER_RETURN ||--o{ ORDER_RETURN_ITEM : order_return_id
    ORDER_ITEM ||--o{ ORDER_RETURN_ITEM : order_item_id
    ORDER_ITEM ||--o{ ORDER_RETURN : "order_item_id (legacy/single-line)"
    STORE_ADDRESS ||--o{ STORE_ADDRESS_USAGE : store_address_id

    JHI_USER ||..o{ CUSTOMER_PROFILE : user_login
    JHI_USER ||..o{ CUSTOMER_ADDRESS : user_login
    JHI_USER ||..o| CUSTOMER_CART : user_login
    JHI_USER ||..o{ WISHLIST_ITEM : user_login
    JHI_USER ||..o{ CUSTOMER_ORDER : user_login
    JHI_USER ||..o{ ORDER_RETURN : user_login
    JHI_USER ||..o{ ORDER_EVENT : user_login
    PRODUCT ||..o{ WISHLIST_ITEM : product_id
    CUSTOMER_ORDER ||..o{ ORDER_EVENT : order_id
    COUPON ||..o{ CUSTOMER_ORDER : coupon_code
    PACKAGE_BOX ||..o{ CUSTOMER_ORDER : package_box_id
    PACKAGE_BOX ||..o{ ORDER_RETURN : package_box_id
```

Core commerce schema

`category`

Column	Type / constraint	Purpose
`id`	bigint PK	Internal identifier
`slug`	varchar(160), unique, not null	Public route key
`name`	varchar(160), not null	Display name
`description`, `image_url`	text / varchar	Content
`sort_order`, `active`	integer / boolean, not null	Merchandising
`zoho_category_id`, `ledgerly_category_id`	varchar, unique	External category mappings

`product`

Primary key: `id`. Foreign key: `category_id → category.id`.

Column group	Important columns
Identity	`slug` unique, `name`, `sku` unique, `category_id`
Description	`fabric_type`, `description`, `story`, `details_kicker`, `badge`
Price/tax	`price`, `compare_at_price`, `currency`, `units`, `hsn`, `tax_pct`
Age/size	`age_min_months`, `age_max_months`, `sizes_json`, `size_chart_json`, `size_notes_json`
Variants/media	`item_structure`, `variants_json`, `colors_json`, `color_images_json`, `images_json`, `zoho_image_docs_json`
Inventory/display	`stock`, `sort_order`, `featured`, `trending`, `new_arrival`, `best_seller`, `active`
Zoho	`zoho_item_id` unique, `zoho_sync_status`, `zoho_last_synced_at`, `zoho_sync_error`
Ledgerly	`ledgerly_item_id` unique, `ledgerly_sync_status`, `ledgerly_last_synced_at`, `ledgerly_sync_error`

Variants and color galleries are serialized JSON fields, not separate relational tables. Code must update them through the application's parsing/validation helpers.

`customer_order`

Primary key: `id`; business key: `order_number` unique. `user_login` is a logical reference to `jhi_user.login`.

Column group	Important columns
Lifecycle	`status`, `payment_status`, `payment_method`, `placed_at`, cancellation fields
Money	`subtotal`, `discount`, `shipping`, `tax`, `donation`, `cod_fee`, `total`, `coupon_code`
Payment	`razorpay_order_id`, `razorpay_payment_id`, `razorpay_signature`
Customer snapshot	`user_login`, `shipping_address_json`, `notes`
Reservation	`inventory_reserved`, `inventory_reserved_until`, `inventory_released_at`
Zoho/Ledgerly	sales-order, invoice, credit-note, PDF, sync-status, error, and tracking-pull fields
Forward shipment	logistics mode, AWB/tracking, label, provider request/status, delivered/cancelled audit fields
Package	weight/dimensions, chargeable weight, courier, `package_type`, logical `package_box_id`, `packed_at`
Short shipment	`package_short_refund_amount`, `package_short_razorpay_refund_id`

`order_item`

Primary key: `id`; enforced foreign key: `order_id → customer_order.id`.

The row is an immutable order-time product snapshot: `product_id`, slug/name/image, size, color, variant SKU, ordered `qty`, `packed_qty`, unit price, MRP, HSN, tax percent, and `discount_allocation`. `product_id` is deliberately not an FK so catalog edits/deletion cannot rewrite historical orders.

`coupon`

Primary key `id`; business key `code` unique. Supports `PERCENT`, `FLAT`, and `BOGO`, with value/minimum/maximum, buy/get quantities, eligibility scope (`ALL`, `SAME_SKU`, `CATEGORY`), optional category id, expiry, and active flag. `customer_order.coupon_code` and `eligibility_category_id` are logical references.

Returns, exchanges, and cancellation

```
erDiagram
    CUSTOMER_ORDER ||--o{ ORDER_CANCELLATION_REQUEST : customer_order_id
    CUSTOMER_ORDER ||--o{ ORDER_RETURN : customer_order_id
    ORDER_RETURN ||--o{ ORDER_RETURN_ITEM : order_return_id
    ORDER_ITEM ||--o{ ORDER_RETURN_ITEM : order_item_id
    ORDER_ITEM ||--o{ ORDER_RETURN : order_item_id

    ORDER_CANCELLATION_REQUEST {
        bigint id PK
        varchar request_number UK
        bigint customer_order_id FK
        varchar user_login
        varchar status
        text reason
        decimal refund_amount
        varchar razorpay_refund_id
    }

    ORDER_RETURN {
        bigint id PK
        varchar return_number UK
        bigint customer_order_id FK
        bigint order_item_id FK
        varchar user_login
        varchar request_type
        varchar status
        decimal refund_amount
        varchar reverse_awb
        varchar outbound_awb
        varchar settlement_status
    }

    ORDER_RETURN_ITEM {
        bigint id PK
        bigint order_return_id FK
        bigint order_item_id FK
        integer requested_qty
        integer received_qty
        integer accepted_qty
        decimal refund_amount
        varchar disposition
    }
```

`order_return_item` has a unique constraint on `(order_return_id, order_item_id)`. It is the current multi-line quantity model. `order_return.order_item_id` remains for compatibility with earlier single-line return/exchange flows.

`order_return` also stores reverse-pickup and replacement-shipment provider fields, package dimensions, carrier states, QC disposition, replacement SKU/size/color, invoice/credit-note ids, exchange price settlement, restock/reservation timestamps, image evidence JSON, and manual refund audit data.

Customer and notification tables

Table	Keys	Purpose
`customer_profile`	PK `id`; unique `user_login`	Name, phone, subscription flag
`customer_address`	PK `id`; index `user_login`	Saved delivery addresses and default flag
`customer_cart`	PK `id`; unique `user_login`	Cart `items_json` and optional `coupon_json`
`wishlist_item`	PK `id`; unique `(user_login, product_id)`	Saved product reference
`order_event`	PK `id`; unique event tuple	Customer notification/activity feed for orders
`contact_message`	PK `id`; index `created_date`	Contact form submission and timestamp

All `user_login` links are logical string references to `jhi_user.login`, with no FK. This allows service separation but means a login rename must be handled by application logic.

Authentication and roles

```
erDiagram
    JHI_USER ||--o{ JHI_USER_AUTHORITY : user_id
    JHI_AUTHORITY ||--o{ JHI_USER_AUTHORITY : authority_name

    JHI_USER {
        bigint id PK
        varchar login UK
        varchar password_hash
        varchar email UK
        boolean activated
        varchar activation_key
        varchar reset_key
        timestamp reset_date
    }

    JHI_AUTHORITY {
        varchar name PK
    }

    JHI_USER_AUTHORITY {
        bigint user_id PK,FK
        varchar authority_name PK,FK
    }
```

`jhi_user_authority` uses a composite primary key. Standard JHipster audit columns are present on `jhi_user`.

Content, operations, and configuration

Table	Business key / relationship	Purpose
`banner`	`id`; ordered by `sort_order`	Hero image, CTA, eyebrow, and trust labels
`blog_post`	unique `slug`	Blog content and publish state
`home_section`	unique `section_key`	Configurable home/shop section JSON
`page_visibility`	unique `page_key`	Page feature switches
`policy`	unique `slug`	Policy documents
`site_setting`	singleton by application convention	Brand, contact, social, shipping, GST, invoice, subscription, about, gift, and policy-window settings
`store_address`	`id`	Operational sender/invoice addresses
`store_address_usage`	FK to `store_address`; unique address/usage	Multi-valued `SHIPMENT`, `NORMAL`, `INVOICE` usage
`package_box`	unique `name`	Reusable package dimensions and tare weight
`zoho_setting`	singleton by application convention	Zoho organisation/OAuth connection state and last product pull
`ledgerly_setting`	singleton by application convention	Ledgerly URL/token/org/connection and webhook configuration

Secrets can exist in the integration setting tables. Database exports must therefore be encrypted and access-controlled even when environment files are excluded.

Webhook idempotency and audit tables

Each table has `id` as its PK, `event_id` as a unique provider/idempotency key, and `received_at`.

Table	Additional identifiers / result fields
`zoho_stock_webhook_event`	`sku`, `product_id`
`zoho_shipment_webhook_event`	Zoho sales order/number, package, shipment, status
`zoho_order_status_webhook_event`	Zoho sales order/number, status
`zoho_item_webhook_event`	Zoho item id, SKU, status
`zoho_invoice_webhook_event`	Zoho invoice/sales order ids, invoice number, status
`xpressbees_shipment_webhook_event`	order number, AWB, provider status, local status
`xpressbees_return_webhook_event`	return number, reverse AWB, provider status, local status
`razorpay_webhook_event`	event type, order number, refund id, local status
`ledgerly_webhook_event`	event type

These rows are operational audit/idempotency records, not the source of truth for the order itself. The canonical state remains in `customer_order` or `order_return`.

Main enumerated values

Field / enum	Values
Order status	`PENDING`, `CONFIRMED`, `HAND_WRAPPED`, `CANCELLED`, `FULFILLED`
Payment status	`PENDING`, `PAID`, `FAILED`, `PARTIALLY_REFUNDED`, `REFUND_INITIATED`, `REFUND_FAILED`, `REFUNDED`
Return status	`REQUESTED`, `APPROVED`, `REJECTED`, `PICKUP_REQUESTED`, `PICKUP_SCHEDULED`, `PICKUP_FAILED`, `PICKED_UP`, `RECEIVED`, `INSPECTED`, `ACCEPTED`, `ITEM_ALLOCATED`, `REPACKING`, `READY_TO_SHIP`, `SHIPPED`, `DELIVERED`, `REFUNDED`, `COMPLETED`, `CANCELLED`, `FAILED`
Request type	`RETURN`, `EXCHANGE`
Logistics mode	`XPRESSBEES`, `MANUAL`, `SELF_SHIP`
Return disposition	`RESTOCKABLE`, `QUARANTINE`, `REPAIR`, `DISCARD`, `RETURN_TO_VENDOR`
Cancellation status	`REQUESTED`, `APPROVED`, `REJECTED`, `WITHDRAWN`
Package type	`CUSTOM`, `BOX`, `ENVELOPE`
Sync status	`PENDING`, `SYNCED`, `FAILED`

Tables managed by infrastructure

Liquibase maintains `databasechangelog` and `databasechangeloglock`. Do not edit these manually. The base JHipster changelog may also create `jhi_date_time_wrapper` for framework/test support; it is not a business table.

Production verification queries

Run with a read-only or appropriately restricted PostgreSQL account. Do not paste credentials into shell history.

```
-- Schema version and most recently executed changes
SELECT id, author, filename, dateexecuted, orderexecuted
FROM databasechangelog
ORDER BY orderexecuted DESC
LIMIT 20;

-- Expected application tables
SELECT table_name
FROM information_schema.tables
WHERE table_schema = 'public'
ORDER BY table_name;

-- Enforced foreign keys
SELECT
    tc.constraint_name,
    tc.table_name,
    kcu.column_name,
    ccu.table_name AS referenced_table,
    ccu.column_name AS referenced_column
FROM information_schema.table_constraints tc
JOIN information_schema.key_column_usage kcu
  ON tc.constraint_name = kcu.constraint_name
 AND tc.constraint_schema = kcu.constraint_schema
JOIN information_schema.constraint_column_usage ccu
  ON ccu.constraint_name = tc.constraint_name
 AND ccu.constraint_schema = tc.constraint_schema
WHERE tc.constraint_type = 'FOREIGN KEY'
  AND tc.table_schema = 'public'
ORDER BY tc.table_name, tc.constraint_name;

-- Column-level definition for any table
SELECT ordinal_position, column_name, data_type, is_nullable, column_default
FROM information_schema.columns
WHERE table_schema = 'public' AND table_name = 'customer_order'
ORDER BY ordinal_position;
```

Expected latest change id: confirm the change from `20261014120000_add_package_short_ship_qty.xml` appears in

`databasechangelog`. Runtime Liquibase is disabled in production, so a missing change must be applied through `deploy/scripts/run-catalog-liquibase.sh` during an approved deployment; do not patch production tables manually.